Programa:	Registry Medic 2.50	
Protección:	SVKP 1.3x + Trial 30 días + Opciones deshabilitadas	
Objetivos:	Desempakar y evitar la limitación	
URL descarga:	www.fu2000.com (creo)	
Dificultad:	No fue fácil…	
Herramientas:	OllyDbg + PEiD + Topo + Import Reconstructor.	
Cracker:	SyXe'05 [hAcK-c0d3d] [CracksLatinos]	Fecha: 28-08-2005

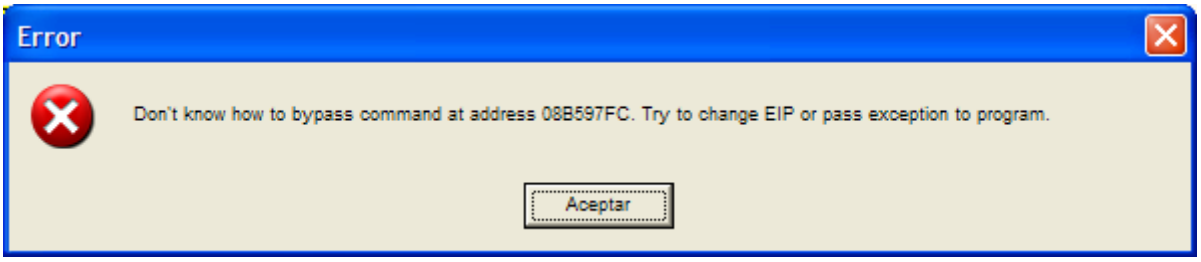
Hola a todos otra vez..!!

Hoy venimos con un programa empaquetado con ***SVKP*** que ya fue resuelto en la lista por el amigo ***Shoulk***, muy bien por cierto sobretodo el método de la ***IAT***, lo que realmente me lanzó a escribir el tute es por varios motivos, el más importante porque utilizaremos un método distinto de reparación de la tabla de importaciones, recuerdo que alguien pidió en la lista una manita con ella, no recuerdo quién era sorry!! Pero si le brindo toda mi ayuda para lo que necesite, no obstante aquí veremos como resolverla, otro de los motivos que me movió a hacer el tut fue que el desempakado de ***Shoulk*** queda perfecto aunque tiene un ligero pero ya que no realiza la función de Reparar el Registro..!! está todo bien menos esto, o sea que lo analiza bien pero al decirle ***Reparar y/o borrar registro*** el programa falla, he de decir que ya tengo el crack final y este tema me costó bastante resolverlo, conseguir que realizara la función no fue nada fácil lo reconozco, inclusive lo dejé por imposible y al día siguiente o sea hoy lo he retomado y he coseguido que la haga, es por eso que escribiré el tut.

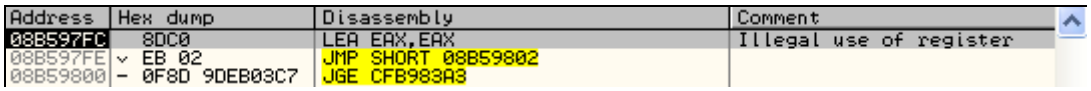
Desempakando SVKP 1.3x de Pavol Cerven…

Lo primero es lo primero, ya sabemos de qué packer se trata porque lo acabo de decir yo pero siempre deberemos cerciorarnos de esto meidante ***PEiD***, ***RDG*** u similar, en este caso ***PEiD*** nos dice esto:

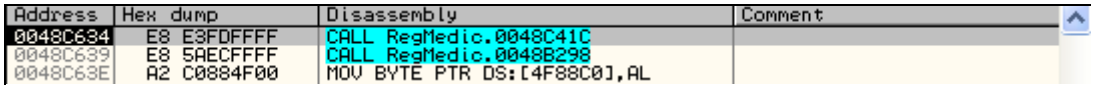
Vale perfecto ya sabemos lo que hay, lo siguiente que haremos será cargar el empakado en ***Olly*** a ver si tiene antidebug, si tildamos las excepciones para que no pare en ellas, ***Olly*** nos avisa que el ***EP*** está fuera de la sección de código, no importa, damos ***RUN*** y para aquí:



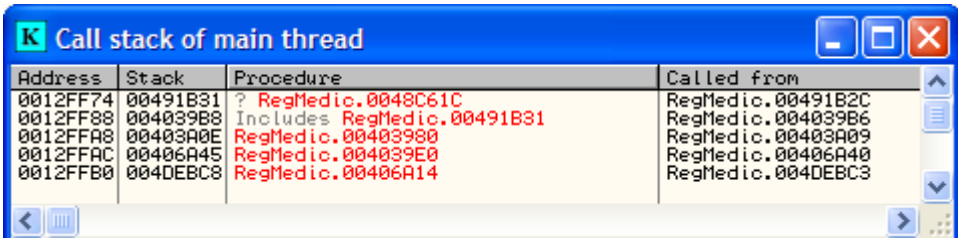
Aceptamos y al dar a ***Shift+F9*** supera la excepción y el programa ya carga, , esto es bueno pues parece ser que ***antidebug*** no tiene, reiniciamos con ***Ctrl+F2*** y destildamos las casillas en el cuadro de excepciones como todos sabemos hacer, si alguien tiene dudas puede revisar el tema en cualquier tut de ***ASProtect*** que en casi todos se explica esto, busquemos el ***OEP***, damos ***RUN*** y superamos todas las excepciones que aparezcan, anotamos la última:



Esta es en mi máquina, repito el proceso y me detengo en ella, antes de superarla pongo un ***BPM On Acces*** en la primera sección y luego si la paso con ***Shift+F9*** como siempre y para aquí:



Que es esto, tiene ***Stolen Bytes***? Si miramos el stack vemos que ya ha ejecutado bastantes instrucciones ¿? Miremos en el ***call stack***:



Pues sí ha ejecutado bstante código es verdad, en concreto estamos dentro de varios calls que se ejecutaron desde el ***OEP*** si nos situamos en la última línea donde el ***RegMedic.00406A14*** y hacemos ***Show Call*** estaremos justo debajo del ***OEP*** que ya hemos pasado, lo que ocurre aquí es que la excepción que sucedió no era la nuestra, deberemos retomarla una antes, que era:

```
Address   Hex dump     Disassembly          Comment
08ADC026    CD 01      INT 1
08ADC028    E8 01000000  CALL 08ADC02E
08ADC02D  - E9 83C4047C  JMP 84B284B5
```

Esta es nuestra última excepción pues, ponemos el ***BPM*** en la primera sección damos ***Shift+F9*** y caemos en esta parte:

```
Address   Hex dump     Disassembly          Comment
08AFB181    AC         LODS BYTE PTR DS:[ESI]
08AFB182    66:50      PUSH AX
08AFB184    3242 20    XOR AL,BYTE PTR DS:[EDX+20]
```

Esto no parece un ***OEP***, si miramos en la barra de estado vemos esto:

Memory breakpoint when reading [00401000]

Nos dice claramente que el motivo de la parada fue por lectura, o sea que está leyendo y no ejecutando, esto no nos interesa, además es un loop, veamos donde termina el mismo:

```
Address   Hex dump     Disassembly          Comment
08AFB20F  ^ E9 6DFFFFFF  JMP 08AFB181
08AFB214    59         POP ECX
08AFB215    5F         POP EDI
08AFB216    61         POPAD
08AFB217    C3         RETN
```

El bucle termina aquí, ponemos un ***BP*** en esta instrucción como se ve en la imagen, antes de dar ***RUN*** quitemos el ***BPM*** que tenemos pues molesta un poco, luego si damos a ***F9*** y paramos en el ***POPAD***, quitamos el ***BP***, volvemos a poner nuevamente el ***BPM*** en la primera sección y al dar ***RUN***…tacháaaaaaaaan… para en el ***OEP***!!!

```
Address   Hex dump     Disassembly          Comment
004DEBB8    55         PUSH EBP
004DEBB9    8BEC       MOV EBP,ESP
004DEBBB    83C4 F4    ADD ESP,-0C
004DEBBE    B8 C8E74D00  MOV EAX,RegMedic.004DE7C8
004DEBC3    E8 4C7EF2FF  CALL RegMedic.00406A14
004DEBC8    A1 D4794F00  MOV EAX,DWORD PTR DS:[4F79D4]
004DEBCD    8B00       MOV EAX,DWORD PTR DS:[EAX]
004DEBCF    E8 F4FCF6FF  CALL RegMedic.0044E8C8
```

Ahí lo tenemos y no hay ***Stolen Bytes***, lo anotamos como siempre:

OEP: 4DEBB8

Y ya que estamos vamos a por la tabla de Importaciones, busquemos los saltos a la ***IAT***, dentro del primer call llegamos a:

```
Address   Hex dump     Disassembly          Comment
00406914  - FF25 58F25000  JMP DWORD PTR DS:[50F258]
0040691A    8BC0       MOV EAX,EAX
0040691C  - FF25 54F25000  JMP DWORD PTR DS:[50F254]
00406922    8BC0       MOV EAX,EAX
```

Estos son dos saltos a la tabla, hay un par más justo aquí, sigamos uno ***Follow in dump->Memory Address*** y caemos en la tabla, buscaremos el inicio de la misma subiendo un poquitín con los cursores:

```
Address   Hex dump                                         ASCII
0050F134  00 00 00 00 00 1A 11 00 EC F8 10 00 00 00 00 00  .....→◄.ÿ°►.....
0050F144  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0050F154  53 9E B6 08 58 9E B6 08 6A 9E B6 08 6F 9E B6 08  S×Ã■X×Ã■j×Ã■o×Ã■
0050F164  74 9E B6 08 79 9E B6 08 87 9E B6 08 95 9E B6 08  t×Ã■y×Ã■ç×Ã■ò×Ã■
0050F174  A7 9E B6 08 B9 9E B6 08 BE 9E B6 08 C3 9E B6 08  º×Ã■╣×Ã■¾×Ã■├×Ã■
```

Este es el inicio que anotamos por supuesto:

Inicio IAT: 50F154

Hagamos lo propio con el final de la misma:

```
Address   Hex dump                                         ASCII
0050F8D4  11 B5 B6 08 16 B5 B6 08 24 B5 B6 08 29 B5 B6 08  ◄ÃÃ■▬ÃÃ■$ÃÃ■)ÃÃ■
0050F8E4  2E B5 B6 08 00 00 00 00 3C B5 B6 08 4E B5 B6 08  .ÃÃ■....<ÃÃ■NÃÃ■
0050F8F4  60 B5 B6 08 72 B5 B6 08 84 B5 B6 08 00 00 00 00  `ÃÃ■rÃÃ■äÃÃ■....
0050F904  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0050F914  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Dentro de la zona sompreada está el último puntero en ***:50F8FC***, por suguridad situemos el final en ***:50F90*** y lo anotamos también:

Final IAT: 50F900

Restando ambos valores obtenemos su largo que también anotaremos:

Largo IAT: 7AC

Nuestro siguiente objetivo será evitar que nos redirija los punteros, vamos a centrarnos en una entrada mala, la primera de ellas será un buen ejemplo, tenemos que en ***[50F154]*** está el valor ***08B69E53***, este será nuestro valor guía al que le haremos un seguimiento, pongamos un ***Hardware BP on Write*** en esta entrada y reiniciemoa todo, tildamos en las casillas de las excepciones para que no pare en ellas, sólo queremos que ***Olly*** pare cuando se escriba en nuestra entrada, pero no para ¿? No importa en lugar de ***Hardware*** lo pondremos ***BPM on Write*** y vemos que ahora sí que para aquí:

```
Address   Hex dump   Disassembly          Comment
08AFB206  AA         STOS BYTE PTR ES:[EDI]
08AFB207  66:58      POP AX
08AFB209  86C4       XCHG AH,AL
```

Va rellenando byte a ***byte*** la entrada que estamos estudiando pero los valores finales fueron ***12 F9 10 00*** que leído al revés queda ***0010F912*** y estos no son los bytes finales por tanto damos ***RUN***, pararemos aquí:

```
Address   Hex dump   Disassembly          Comment
08AF7389  8907       MOV DWORD PTR DS:[EDI],EAX
08AF738B  7C 03      JL SHORT 08AF7390
08AF738D  EB 03      JMP SHORT 08AF7392
```

Aquí sí tiene ***EAX*** los bytes buscados, o sea que es donde guarda los bytes finales, traceemos un poco por la zona a ver como va rellenando la tabla, todo siempre con ***F7*** para ir entrando en los call, nos llama tremendamente la atención que al superar el siguiente POPAD tenemos un valor API en el EAX, veamos el POPAD primero:

```
Address   Hex dump   Disassembly          Comment
08AF7392  61         POPAD
08AF7393  E8 01000000  CALL 08AF7399
08AF7398  9A 83C404EB 02  CALL FAR CD02:EB04C483   Far call
```

Además estamos poco más abajo del lugar donde guarda los valores en la tabla, en este caso quedó esta api:

```
Registers (MMX)
EAX 77E5A7DF kernel32.GetCurrentThreadId
ECX 77E5B5E1 kernel32.77E5B5E1
EDX 00400000 ASCII "MZP"
EBX 0050F914 RegMedic.0050F914
ESP 0012FF94
EBP 08AD0000
ESI 0050F000 RegMedic.0050F000
EDI 0050F154 RegMedic.0050F154
```

Ahí vemos que dejó en ***EAX*** la api ***GetCurrentThreadId***, lo que no sabemos si esta es la api que debería haber guardado o el siguiente, podríamos atacar por aquí, la zona está algo ofuscada, desofusquémosla un poco:

```
Address   Hex dump   Disassembly          Comment
08AF7389  8907       MOV DWORD PTR DS:[EDI],EAX
08AF738B  90         NOP
08AF738C  90         NOP
08AF738D  90         NOP
08AF738E  90         NOP
08AF738F  90         NOP
08AF7390  90         NOP
08AF7391  90         NOP
08AF7392  61         POPAD
08AF7393  E8 01000000  CALL 08AF7399
```

Se ve muchísimo mejor donde guarda y el ***POPAD*** final que deja en ***EAX*** la api que no sabemos con certeza si es la actual o la de la siguiente entrada, probaremos primero lo más sencillo, la trataremos como la entrada actual que debería haber guardado, además al desofuscar la zona nos quedaron 7 bytes donde podemos ensamblar un salto a una zona libre don de injertar una rutinita que guarde la ***API*** correcta en la ***IAT***, el salto ocupará 5 bytes, el ***POPAD*** lo nopearemos aquí y lo ejecutaremos en nuestra rutina, espacio libre tenemos a partir de ***:8AFBE60***, modifiquemos esta zona forzando el salto hacia allí:

```
Address   Hex dump   Disassembly          Comment
08AF7389  8907       MOV DWORD PTR DS:[EDI],EAX
08AF738B  E9 D04A0000  JMP 08AFBE60
08AF7390  90         NOP
08AF7391  90         NOP
08AF7392  90         NOP
08AF7393  E8 01000000  CALL 08AF7399
```

Los bytes asociados al cambio son: 89 07 E9 D0 4A 00 00 90 90 90

Esto nos ayudará por si hay que reescribir la zona simplemente con un ***Binary Paste*** tenemos todo en su lugar ***;-)***
Así es como debe quedar, el ***POPAD*** como se aprecia lo hemos eliminado, ahora en ***:8AFBE60*** injertaremos esto:

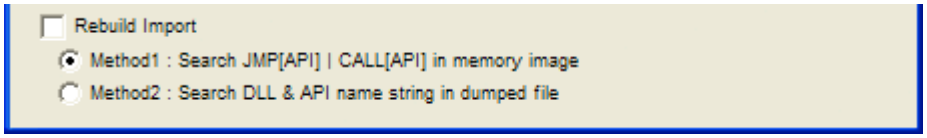
```
Address   Hex dump   Disassembly          Comment
08AFBE60  61         POPAD
08AFBE61  8907       MOV DWORD PTR DS:[EDI],EAX
08AFBE63  81FF FCF85000  CMP EDI,50F8FC
08AFBE69  74 05      JE SHORT 08AFBE70
08AFBE6B  E9 20B5FFFF  JMP 08AF7390
08AFBE70  90         NOP
08AFBE71  90         NOP
```

Bytes asociados: 61 89 07 81 FF FC F8 50 00 74 05 E9 20 B5 FF FF 90 90

No olvidar poner el ***BP*** en ***:08AFBE70*** para que ***Olly*** pare allí cuando termine, la condición de salida es el ***final*** de la ***IAT***, cuando trate el último puntero saltará hacia el ***NOP*** donde está el ***BP*** puesto, situamos el ***EIP*** sobre el ***POPAD*** del injerto con ***New origin Here*** y damos ***RUN***, parará en el ***BreakPoint*** al momento, es hora de ver cómo quedó la nueva tabla después de pasarle el injerto:

```
Address   Hex dump                                          ASCII
0050F164  45 A7 E5 77 CB 15 E6 77 72 AC E5 77 A0 60 E5 77  E§ůw╦§µwr¬ůwá`ůw
0050F174  82 A6 E5 77 71 A6 E5 77 60 A6 E5 77 44 7F E5 77  é¦ůwq¦ůw`¦ůwD⌂ůw
0050F184  49 A9 E5 77 FC A7 E5 77 E1 60 E5 77 4E 27 E5 77  I⌐ůw³§ůwß`ůwN'ůw
0050F194  0A 4E E5 77 41 D9 E5 77 41 DA E5 77 7E 17 E4 77  .NůwA┘ůwA┌ůw~↨ůw
0050F1A4  32 B3 E5 77 F2 33 B5 08 A9 AD E5 77 62 73 E5 77  2│ůw=3A◘⌐↓ůwbsůw
0050F1B4  02 15 F4 77 19 2A B5 08 8C E6 E5 77 0D A1 E5 77  ☺§¶w↓*A◘îµůw.¡ůw
0050F1C4  9D EE E5 77 85 20 B5 08 8F 4A E5 77 53 BE E5 77  Ø¯ůwà A◘AJůwS¥ůw
0050F1D4  3A F1 E5 77 C0 30 E7 77 2E F0 E5 77 26 C6 E4 77  :±ůw└0╠w.-ůw&╠ůw
0050F1E4  44 0C F5 77 4E AB E5 77 37 38 E5 77 94 E4 E5 77  D.♂wN½ůw78ůwöäůw
0050F1F4  64 16 E6 77 FB E3 E5 77 76 B4 E5 77 F0 A6 E5 77  d▬µw'ÔůwvÅ┤ůw-¦ůw
0050F204  00 00 00 00 76 8A D3 77 00 6E D1 77 76 64 D3 77  ....vèËw.nÐwvdËw
0050F214  10 A3 D1 77 00 00 00 00 10 24 DA 77 9A 22 DA 77  ►úÐw....►$┌wÜ"┌w
0050F224  D8 17 DA 77 00 00 00 00 E2 19 0F 77 B8 AB 0F 77  ï↨┌w....ô↓*w@½*w
0050F234  EC 14 0F 77 8F 3D 0F 77 4B 17 0F 77 2E F4 0F 77  ÿ¶*wÅ=*wK↨*w.¶*w
0050F244  74 16 0F 77 00 00 00 00 4F AA E5 77 F1 AB E5 77  t▬*w....O¬ůw±½ůw
0050F254  82 A6 E5 77 F2 33 B5 08 A9 AD E5 77 00 00 00 00  é¦ůw=3A◘⌐↓ůw....
0050F264  B1 63 DB 77 10 24 DA 77 F4 6C DB 77 9A 22 DA 77  ▒c╛w►$┌w¶l╛wÜ"┌w
0050F274  6D 69 DB 77 68 6A DB 77 8B 6F DB 77 D4 65 DB 77  mi╛whj╛wïo╛wÊe╛w
0050F284  E2 68 DB 77 D6 27 DA 77 D8 17 DA 77 00 00 00 00  ôh╛wï'┌wï↨┌w....
0050F294  0A 4E E5 77 7E 8D E4 77 3A F1 E5 77 12 AC E5 77  .NůwÅ~ïäw:±ůw↕¬ůw
0050F2A4  44 7F E5 77 9E 16 E4 77 72 AC E5 77 EA 1B E4 77  D⌂ůwx▬ůwr¬ůwÜ←ůw
0050F2B4  CF D2 E4 77 5D 26 E5 77 A6 9D E5 77 2E F0 E5 77  ╧╥äw]&ůw¦¥ůw.-ůw
0050F2C4  37 5E E5 77 4C EF E5 77 26 C6 E4 77 B3 29 E5 77  7^ůwL'ůw&╠ůw│)ůw
0050F2D4  2D A1 E4 77 1F 5E E5 77 4E AB E5 77 7F 61 E5 77  -¡äw▼^ůwN½ůw⌂aůw
0050F2E4  FC A7 E5 77 A1 39 E5 77 51 E3 E5 77 8B 63 E5 77  ³§ůwí9ůwQôůwïcůw
```

Pues quedó bastante bien, el injerto fue bastante bien aunque quedaron ***10 entradas*** sin resolver, a estas alturas la solución va desde tracear las entradas a mano como ya hizo ***Shoulk*** en su tutorial o bien crear un injerto que cargue la zona del ***packer*** y ejecute allí al código de cada entrada, haremos esto último, y esto se puede hacer almenos de dos maneras distintas, o bien cargando la zona desde un archivo o creando con ***Topo*** una nueva sección donde guardaremos los bytes de la zona, lo haremos con ***Topo***, si alguien quiere saber como se hace del otro modo me puede escribir y encantado se lo diré pues no es difícil., vale pero antes de esto necesitaremos un dumpeado, seleccionamos el injerto y le hacemos un ***Undo Selection*** para que no quede rastro por un posible ***CRC***, lo mismo con el salto que pusimos dejando todo tal cual estaba, situamos el ***EIP*** en :***08AF7393*** con ***New origin Here*** ya que el ***POPAD*** anterior no se debe ejecutar ya que esto lo hizo en el injerto, ponemos un ***BPM*** y llegamos el ***OEP*** como antes hicimos, desde allí dumpeamos el archivo con ***OllyDump***

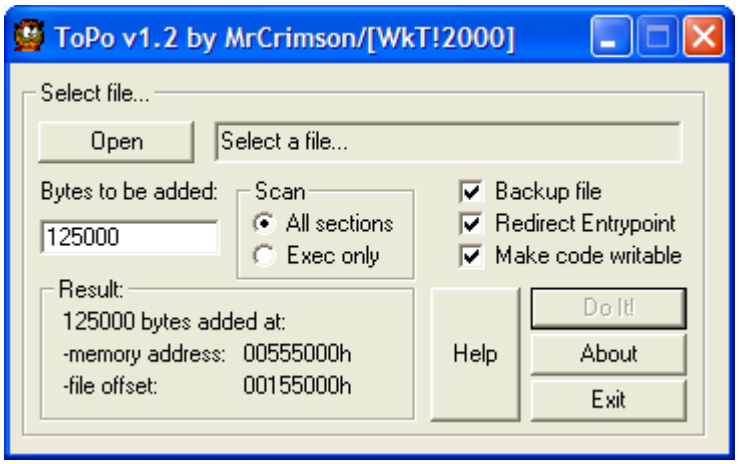


Lo configuramos de esta manera, ya le repararemos la ***IAT*** después nosotros

Una vez tengamos el dumpeado nos centraremos enlas 10 entradas quedaron sin resolver, el prmer puntero quedó en ***[50F1A8]=08B533F2***, otro más fue ***[50F1B8]=08B52A19*** y así hasta 10. Esto lo superaremos con un ionjertito que nos cargue la zona de memoria donde apuntan las diez entradas, veamos esta zona:

00740000	00106000				Map	R E	R E
08B50000	0001D000				Priv	RW	RW
08B70000	00001000	OLE32		PE header	Imag	R	RWE

Crearemos una nueva sección con ***Topo*** en el dumpeado, vemos que el tamaño de la sección es ***1D000h*** que pasado a decimal son ***118784d*** bytes, crearemos la sección con ***125000*** bytes para guardar los bytes y el injerto, mejor que sobre un poco, selegiremos la opción ***Create a new section (file size increased)***:



Así deberemos crearla tal como se ve en la imagen, con esas opciones, la sección que nos falta se deberá cargar a partirn de ***:08B50000*** y deberá tener un tamaño de ***1D000***, picamos sobre la sección con el botón derecho y elegimos ***Dump in CPU***, vamos después a la ventana ***Dump***, seleccionamos todos los bytes y luego ***Binary Copy***, abrimos el dumpeado con la nueva sección en ***Olly*** y vamos a ***:556000*** donde pegaremos los bytes de la sección con ***Binary Paste***, allí debería haber ***NOP'S*** en toda la sección, quedará así:

```
Address   Hex dump                                          ASCII
00555FE0  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
00555FF0  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
00556000  E9 35 01 00 00 00 00 B5 08 00 00 00 00 0C 00 00  Θ50....A◘.......
00556010  00 00 00 00 00 00 00 00 00 8B 68 AD 25 66 D7 AD  ..........ïh↓%fï↓
00556020  25 B9 E2 A1 25 00 00 00 00 12 17 AD 25 00 00 00  %╣ôí%....↨↕↓%...
```

Lo que se ve rojo son los nuevos bytes copiados, depués en la ventana de código vamos a ***:555000*** donde injertaremos nuestra rutina que cargue la sección, pero antes deberemos realizarlo, necesitamos conocer la posición de ***VirtualAlloc*** en la ***IAT***, no es difícil encontrarla,

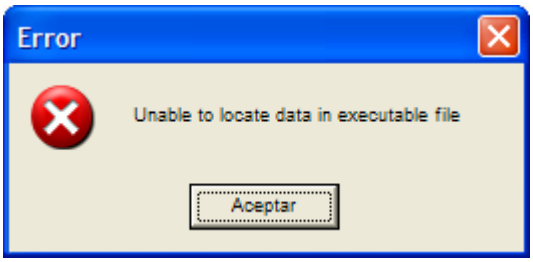
está en ***[50F16C]*** esto es para que pueda funcionar en totas las máquinas no debemos hacer referencia explícita a la api sino a la ***IAT*** donde estará en cualquier ***PC***. La sección se cargará a partir de ***:08B50000*** como sabemos y tendrá un tamaño de ***1D000h*** bytes, sabiendo esto ahora sí podemos crear el injerto que podrá ser este por ejemplo:

```
Address   Hex dump     Disassembly                          Comment
00555000  60          PUSHAD
00555001  6A 04       PUSH 4
00555003  68 00300000 PUSH 3000
00555008  68 00D00100 PUSH 1D000
0055500D  68 0000B508 PUSH 8B50000
00555012  FF15 6CF15000 CALL DWORD PTR DS:[50F16C]         kernel32.VirtualAlloc
00555018  85C0        TEST EAX,EAX
0055501A  BE 00605500 MOV ESI,borrar.00556000
0055501F  BF 0000B508 MOV EDI,8B50000
00555024  B9 00740000 MOV ECX,7400
00555029  F3:A5       REP MOVS DWORD PTR ES:[EDI],DWORD PTR D
0055502B  61          POPAD
0055502C  68 B8EB4D00 PUSH borrar.004DEBB8                 ASCII "Uïÿä-¶Ø⁴ÞM"
00555031  C3          RETN
00555032  90          NOP
```

Lo que hace es muy sencillo, primero salvamos los registros con el ***PUSHAD***, mejor loexplico un poco:

PUSHAD	**; Salvamos los registros**
PUSH 4	**; Código Ejecutable, permite Lectura/Escritura**
PUSH 3000	**; MEM_COMMIT, o sea pedimos que cree la zona ya**
PUSH 1D000	**; Tamaño de la misma**
PUSH 8B50000	**; A partir de :08B50000**
CALL DWORD PTR [50F16C]	**; Llamada a VirtualAlloc**
TEST EAX, EAX	**; Sin interés**
MOV ESI, 556000	**; En ESI la dirección donde están los bytes guardados**
MOV EDI, 8B50000	**; En EDI donde los copiaremos**
MOV ECX, 7400	**; En ECX el tamaño / 4 ya que va de dword en dword**
REP MOVS DWORD PTR [EDI], DWORD PTR [ESI]	**; Copiamos de [ESI] a [EDI] los bytes**
POPAD	**; Restauramos los registros**
PUSH 4DEBB8	
RETN	**; Y saltamos al OEP…**

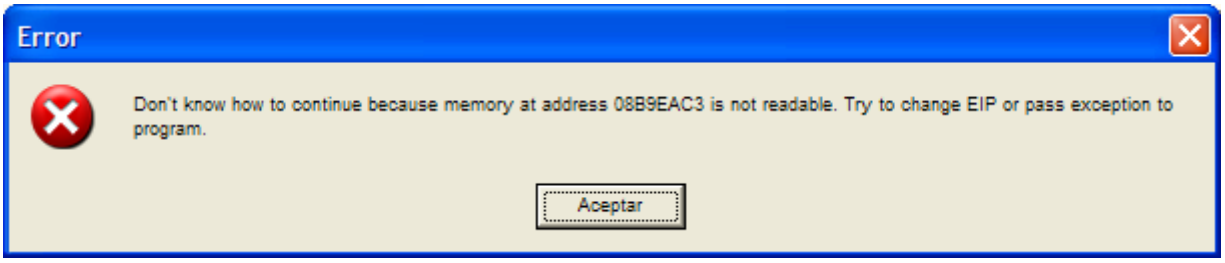
Creo que no merece más comentarios pues es muy sencillo, cuando lo tengamos listo seleccionamos los bytes modificados de la nueva sección con cuidado de no pasarse sino ***Olly*** mostrará este mensaje de error:



Para solventarlo seleccionaremos justo los bytes modificados, ni uno más ui uno menos, veamoslo en la imagen siguiente:

```
Address   Hex dump                                         ASCII
00572FD0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00572FE0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00572FF0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00573000  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
00573010  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉÉ
```

Ahora si podemos salvar los cambios, lo llamaremos ***tut_injerto.exe*** para referirnos a él del mismo modo todos, una vez salvados los cambios cargamos el programa en ***Olly*** traceamos en injerto y vemos que carga bien la sección y que la rellena perfectamente con sus correspondientes bytes, perfecto damos ***RUN*** y…



Bah! No pasa nada en el original también aparece el cartel, lo aceptamos y nos cierra el Olly ¿?

Y ahí nos deja tirados sin nada que hacer, es momento de ir a tracear en busca del error, nuestro objetivo es localizar y destruir la protección, en el original no pasa y eso me hace pensar en algún truquito antidump, tipo comparación del tamaño del archivo, etc… el nombre no es pues lo renombré como el original y nada seguía todo igual, así que cargué el dumpeado en Olly y comencé a tracear:

Address	Hex dump	Disassembly	Comment
00491B22	E8 21E0FBFF	CALL borrar1.0044FB48	Aqui falla..!!
00491B27	E8 24ACFFFF	CALL borrar1.0048C750	
00491B2C	E8 EBAAFFFF	CALL borrar1.0048C61C	

El primero de los tres calls es el que nos trae problemas, si vamos al original y tomamos una instantánea de los registros antes y después del paso por el call vemos que son totalmente iguales o sea que no ejecuta ninguna ***API*** ya que no retornó nngún valor en ***EAX***, la única variación está en el stack donde nos metió dos valores nuevos, probé a nopear el primer call en el dumpeado ejecutar el de el medio y a nopear el siguiente ya que nos da el mismo error, la única diferencia en los registros fue:

Antes:	Después:
ECX=1	ECX=0
EDX=cadena_rara	EDX=mi_nombre

Está claro que no ejecutan ninguna ***API*** estos dos calls y sólo nos dan problemas, dentro del primero de estos hay un salto a la rutina problemética que está en la zona que hemos cargado con el injerto, sustituiremos el salto por un ***RETN***, quedarán pues:

Address	Hex dump	Disassembly	Comment
0044FB48	C3	RETN	
0044FB49	90	NOP	
0044FB4A	90	NOP	
0044FB4B	90	NOP	
0044FB4C	90	NOP	
0044FB4D	90	NOP	

Esto para el primero, para el segundo será diferente ya que apunta a una zona con varios ***CALL***, deberemos encontrar el salto y sustituiro por un ***RETN***, o sea ***RETNarlo***, jeje..

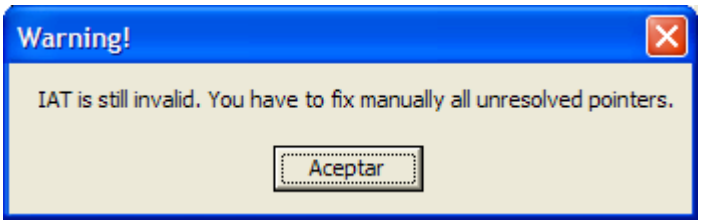
Address	Hex dump	Disassembly	Comment
0044FB38	C3	RETN	
0044FB39	90	NOP	
0044FB3A	90	NOP	
0044FB3B	90	NOP	
0044FB3C	90	NOP	
0044FB3D	90	NOP	

Escondidito que estaba el condenao, aquí estaba el salto, le hicimos lo mismo que al otro, pero todavía no cargaaaaaa el programa, bouuuuuuuff..!! esto nunca acaba, el error se produce en un ***CALL*** que hay en ***:48B15E***, pongamos un ***BP*** en el call, antes salvaremos los cambios que hemos hecho hasta ahora para que no molesten más y trabajar más tranquilos, hecho esto cargamos el programa en ***Olly*** y damos ***RUN***, paramos en el ***BP*** que hemos puesto, entramos en el ***CALL*** con ***F7*** y no hace falta buscar mucho porque allí mismo vemos el salto indirecto a la rutina problemática, lo ***RETNeamos*** también, jejeje..

Address	Hex dump	Disassembly	Comment
0044FB40	C3	RETN	
0044FB41	90	NOP	
0044FB42	90	NOP	
0044FB43	90	NOP	
0044FB44	90	NOP	
0044FB45	90	NOP	

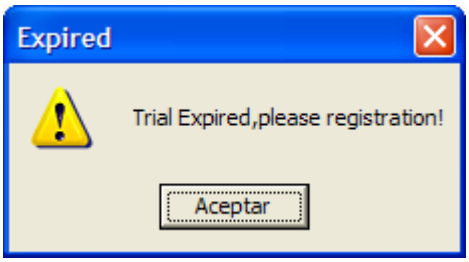
Ya tenemos tres saltos modificados, lo vamos anotando todo para llevar las cuentas de lo que vamos haciendo, salvamos los cambios nuevamente como ***tut_injerto1.exe*** ejecutamos el archivo y… peta todavía debido a que no le hemos reparado la ***IAT*** del todo, debemos ir al ***Import*** y reparla, le damos los siguientes datos:

Al darle a ***Get Imports*** obtendremos ***10*** entradas que son las que no pudo reparar el injerto que hicimos, bueno no son muchas, de momento reparemos las que tenemos, el resto veremos si se ejecutan bien en la zona del ***packer*** que hemos cargado, veremos este mensaje:

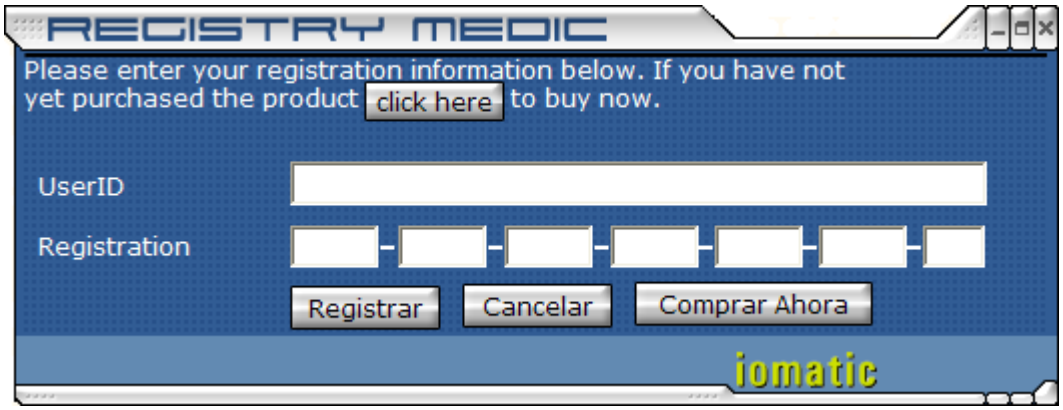


Indica que todavía quedan punteros sin resolver que son los ***10*** que ya sabemos, nada aceptamos, nos habrá creado un nuevo ejecutable con la ***IAT semi-reparada***, este es el que debemos cargar ahora en ***Olly*** y tracear un poco, se llama ***tut_injerto1_.exe*** y…

Aparece el siguiente mensaje:



y justo detrás este cuadro de registro:



Todo esto me hace pensar que el packer está ***K.O***. y que es de la protección propia del programa. No suelo hacerlo pero ya llevamos casi ocho pags. con letra tamaño ***9*** que es bastante y cono el ***Trial*** me llevó más trabajito que el ***packer*** y no es broma lo dejaremos para la segunda parte del tut donde veremos como superar el duro ***Trial*** del programa.

SyXe'05
[hAcK-c0d3d]
[CracksLatinos]